MODULE M15, CONDUITE DE PROJET AI ENGINEERING

Rapport de conduite de projet AI Engineering

Observabilité et garde-fous (juge-LLM) du triage médical CHSA

Auditer, instrumenter et superviser un agent IA de pré-triage, jusqu'à ses garde-fous de sécurité, sans jamais le modifier

par **Rafael CEREZO MARTIN**
AI Engineer Openclassrooms
Octobre 2026

Tabla de Contenidos

1. Contexte et analyse des besoins **2**
1.1 Présentation (organisation et / ou contexte) 2
1.2 Collecte et analyse du besoin métier 2
2. Audit de la solution data existante **4**
2.1 Solution actuelle et proposée 4
2.2 Évaluation de l'adéquation aux besoins 5
3. Identification d'une solution technique cible **6**
Comparatif d'approches techniques 6
Avantages/inconvénients. 6
Schéma d'architecture cible clair 6
Justification des choix technologiques 7
Méthodologie d'identification, d'évaluation et de priorisation des cas d'usage 7
Hiérarchisation documentée des cas d'usage 7
4. Stratégie de mise en œuvre et d'industrialisation **8**
4.1 Proposition de démarche projet 8
4.2 Aide à la prise de décision 9
5. Contrôle et suivi du projet **9**
5.1 Tableau de bord de pilotage 9
5.2 Outils et process de suivi 10
6. Conclusion & recommandations **11**
Résumé des choix clés 11
Perspectives d'évolution. 11
Prochaines étapes recommandées. 11
7. Annexes (Réferences) **11**

Glossaire

Terme	Définition
IA générative	Classe de modèles d'intelligence artificielle capables de produire du texte, du code ou d'autres contenus à partir d'un contexte donné.
LLM (Large Language Model)	Modèle de langage de grande taille, entraîné sur de vastes corpus de texte, capable de comprendre et de générer du langage naturel.
Fine-tuning	Processus d'adaptation d'un modèle pré-entraîné à une tâche ou un domaine précis, à partir d'un jeu de données restreint.
SFT (Supervised Fine-Tuning)	Fine-tuning supervisé : le modèle apprend à partir de paires question/réponse annotées.
DPO (Direct Preference Optimization)	Méthode d'alignement d'un modèle sur des préférences humaines, par comparaison directe de réponses préférées et rejetées.
LoRA / PEFT	Techniques de fine-tuning n'entraînant qu'un petit nombre de paramètres additionnels, pour réduire fortement le coût de calcul.
Quantification (GGUF, Q4_K_M)	Réduction de la précision numérique des poids d'un modèle pour diminuer son empreinte mémoire et accélérer l'inférence.
Inférence	Étape d'utilisation d'un modèle entraîné pour produire une réponse à partir d'une nouvelle entrée.
API (Application Programming Interface)	Interface permettant à deux systèmes logiciels de communiquer selon un contrat défini.
Architecture hexagonale	Style d'architecture isolant la logique métier des dépendances techniques externes via des ports (interfaces) et des adaptateurs (implémentations).
Observabilité	Capacité à comprendre l'état interne d'un système à partir des données qu'il expose : métriques, journaux, traces.
Métrique	Mesure numérique collectée dans le temps, utilisée pour suivre le comportement d'un système.
Prometheus	Système de collecte et de stockage de métriques de supervision, interrogé via le langage PromQL.
Grafana	Outil de visualisation permettant de construire des tableaux de bord à partir de sources comme Prometheus.
PromQL	Langage de requête de Prometheus, utilisé pour agréger et transformer les métriques collectées.
Tableau de bord de pilotage	Ensemble de visualisations synthétiques permettant de suivre en continu l'état et la performance d'un système.
Journal d'audit	Fichier structuré consignant les événements significatifs d'un système, utilisé comme source de vérité pour l'analyse a posteriori.
Triage médical	Processus clinique de hiérarchisation des patients selon l'urgence de leur état, en amont de leur prise en charge.
Niveau ESI (Emergency Severity Index)	Échelle standardisée à cinq niveaux utilisée aux urgences pour classer l'urgence clinique d'un patient.
MLOps	Ensemble de pratiques visant à industrialiser et fiabiliser le cycle de vie des modèles d'apprentissage automatique.

Tableau , Glossaire des sigles et termes techniques utilisés dans ce rapport.

1. Contexte et analyse des besoins

Ce projet ne se déroule pas au sein d'une entreprise au sens classique : il prolonge un projet de certification AI Engineer déjà validé, CHSA Triage (ci-après « le système audité »), un assistant d'intelligence artificielle d'aide au pré-triage médical. Le présent rapport traite la suite logique de ce premier projet : sa mise sous contrôle opérationnelle, condition nécessaire avant d'envisager un déploiement en conditions réelles. Le contexte « organisation » est donc celui, réaliste, d'un service d'urgences qui envisagerait d'adopter cet assistant comme outil d'aide à la décision pour son personnel infirmier.

1.1 Présentation (organisation et / ou contexte)

Le système audité se positionne comme un outil d'aide, jamais de substitution : il recueille les symptômes déclarés par le patient au fil d'un court entretien, puis propose un diagnostic structuré (niveau d'urgence, catégorie clinique, ressources estimées) que l'infirmier reste seul habilité à valider ou à écarter.

Secteur d'activité, taille de l'organisation, enjeux business.

Le secteur visé est celui des urgences hospitalières, un environnement où le temps avant la première évaluation clinique est un facteur déterminant de la qualité de la prise en charge.

- **Secteur** : santé, accueil et orientation des urgences.
- **Utilisateur final** : personnel infirmier d'accueil (IOA).
- **Échelle visée** : un outil de pré-triage en phase pilote, destiné à un seul service avant toute généralisation.
- **Enjeu business** : réduire le délai avant la première évaluation, sans jamais retirer la décision finale au personnel soignant.

Niveau de maturité IA / MLOps.

Le système audité dispose déjà d'un modèle fiabilisé et reproductible, mais reste, au sens MLOps, à un stade antérieur à toute mise en production surveillée.

- **Modèle Qwen3-1.7B affiné** par apprentissage supervisé (SFT) puis par optimisation directe des préférences (DPO) sur des cas de triage.
- **Quatre configurations** de déploiement déjà disponibles, du poste local gratuit à l'hébergement distant sur GPU.
- **Architecture hexagonale** testée, donc une base de code reproductible et maintenable.
- **Aucune supervision en continu** : le comportement du système en usage réel reste aujourd'hui invisible une fois le modèle déployé.

Contraintes technologiques, de sécurité, d'infrastructure ou de scalabilité.

Quatre familles de contraintes encadrent strictement la façon dont ce projet peut intervenir sur le système existant.

- **Contrainte technologique** : le système audité est déjà validé en certification ; il doit rester inchangé, à l'octet près.
- **Contrainte d'infrastructure** : les quatre configurations de déploiement sont hétérogènes (CPU local, GPU distant) et doivent être auditées indépendamment, sans confusion entre elles.
- **Contrainte de sécurité** : les échanges patient/assistant sont traités comme des données sensibles ; aucun contenu textuel n'est dupliqué hors de son usage prévu.
- **Contrainte de scalabilité** : plusieurs configurations peuvent être auditées en parallèle ; leurs métriques doivent rester cloisonnées, jamais mélangées.

1.2 Collecte et analyse du besoin métier

En l'absence d'une organisation en exploitation réelle, le besoin a été constitué à partir de deux sources convergentes : la décision de périmètre que le système audité documente lui-même dans son propre code, et une exploration technique directe de son comportement réel.

Parties prenantes impliquées

Quatre parties prenantes aux intérêts distincts, parfois en tension (fiabilité clinique perçue contre rigueur méthodologique, par exemple), ont structuré les arbitrages du projet :
Quatre parties prenantes types ont été identifiées pour cadrer le besoin :

Partie prenante	Intérêt principal
Personnel infirmier d'accueil	Disposer d'un outil fiable, sans surcharge de saisie ni fausse confiance.
Équipe IA / data	Garantir la qualité et la reproductibilité du modèle dans la durée.
Responsable sécurité / conformité	S'assurer qu'aucune donnée sensible n'est exposée par l'audit.
Jury / mentor de certification	Évaluer la rigueur de la démarche de conduite de projet.

Recueil du besoin

Le recueil du besoin n'a pas pris la forme d'entretiens classiques, faute d'organisation réelle à interroger. Il s'est appuyé sur une méthode d'investigation directe du système existant, assimilable à un recueil technique de besoin :

- Lecture du code source et de sa documentation interne, qui assume explicitement l'absence de toute supervision (« jamais une stack d'observabilité lourde »).
- Exploration active du système par des appels réels à son interface, pour observer son comportement effectif plutôt que son comportement supposé.
- Confrontation des deux : un système reproductible mais opaque une fois en usage, ce qui définit le besoin central du projet.

Objectifs techniques et/ou business visés

Les objectifs retenus se répartissent en deux registres complémentaires :

- **Objectif technique** : construire un pipeline de collecte, d'exposition et de visualisation des métriques, sans modifier le système audité.
- **Objectif business** : fournir aux décideurs un tableau de bord objectif pour arbitrer entre les quatre configurations de déploiement (coût, performance, fiabilité).
- **Objectif qualité** : rendre visibles les anomalies de fonctionnement réelles, aujourd'hui noyées dans un simple fichier journal.

Contraintes réglementaires, éthiques, de sécurité ou de déploiement.

Un système d'aide à la décision clinique impose une vigilance particulière, même en contexte de certification :

- **Traçabilité** : chaque échange doit rester auditable a posteriori, d'où l'intérêt central du journal d'audit déjà présent côté système.
- **Sécurité d'accès** : toute interaction avec le système audité est protégée par une clé d'API.
- **Déploiement** : la coexistence de plusieurs configurations impose un étiquetage explicite de chaque métrique par sa configuration d'origine, pour ne jamais produire un chiffre ambigu.

Hiérarchisation des besoins

Les besoins d'observabilité identifiés ont été classés selon une matrice impact/effort, afin de concentrer l'effort initial sur les mesures à forte valeur et faible coût d'implémentation, et de documenter explicitement ce qui est reporté plutôt que de l'ignorer.

Besoin identifié	Impact	Effort	Décision
Volume, latence et taux d'erreur des échanges	Élevé	Faible	Réalisé en priorité
Conformité du format de diagnostic	Élevé	Faible	Réalisé
Agrégats au niveau d'une conversation (nombre de tours, délai, échecs)	Moyen	Moyen	Réalisé
Pertinence clinique jugée par un modèle tiers	Élevé	Élevé	Reporté (travaux futurs)
Garde-fou de sécurité/forme jugé par un modèle tiers (juge-LLM dédié)	Élevé	Moyen	Réalisé (F5), distinct de la ligne ci-dessus : juge la forme/sécurité (recommandation de consulter, absence de langage absolu, absence de donnée personnelle), jamais l'exactitude clinique
Niveau d'urgence (ESI) en continu	Élevé	Élevé	Reporté, nécessite une évolution du système audité

Critères de cotation de la matrice

La cotation n'a pas été intuitive : chaque niveau répond à une règle explicite, afin que la même grille puisse être réappliquée à tout nouveau besoin.

Impact

- **Élevé** : le besoin touche directement à la sécurité du patient (format du diagnostic, garde-fou) ou conditionne un arbitrage de déploiement (coût, choix de configuration).
- **Moyen** : le besoin améliore la compréhension du système sans conditionner, à lui seul, une décision.

Effort

- **Faible** : la donnée figure déjà dans chaque entrée du journal d'audit ; un simple compteur ou histogramme suffit.
- **Moyen** : il faut un état conservé entre plusieurs entrées (accumulateur de conversation) ou un composant local supplémentaire (juge-LLM).
- **Élevé** : la donnée n'est jamais consignée par le système audité. Pour l'obtenir, il faudrait le modifier, ce que la contrainte « inchangé à l'octet près » interdit dans ce projet. Il peut aussi falloir un référentiel médical externe.

Conséquence directe. Le niveau ESI n'est pas reporté parce qu'il serait peu utile, mais parce que son coût réel est une modification du système audité. Le report est donc imposé par une contrainte, et non choisi par commodité.

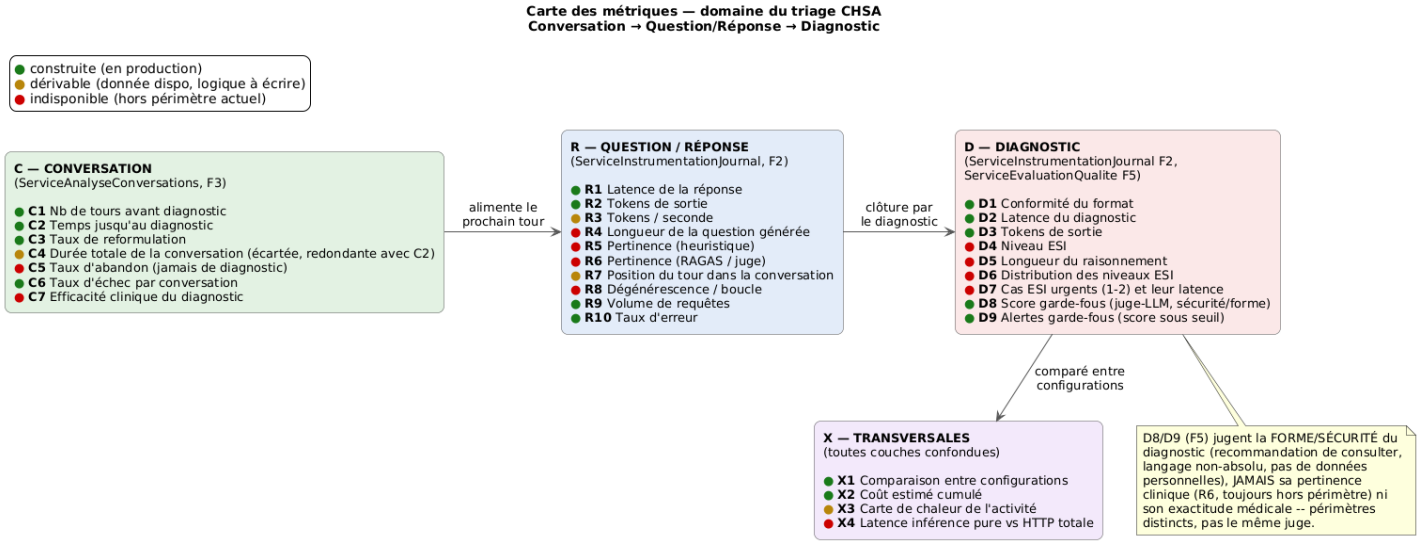


Figure 1. Carte des métriques d'observabilité par niveau de domaine, classées par statut (construite, dérivable ou hors périmètre), support de la hiérarchisation ci-dessus.

Lecture de la carte des métriques

La carte organise les mesures selon quatre niveaux de granularité. Une même grandeur ne signifie pas la même chose selon l'unité observée : la latence d'un tour (R1) et celle d'un diagnostic (D2) relèvent de deux attentes cliniques distinctes.

- **R, Question / Réponse** : l'unité est l'échange élémentaire. Chaque entrée du journal produit directement ces mesures, sans mémoire d'état (F2).
- **C, Conversation** : l'unité est l'entretien complet. Il faut agréger plusieurs entrées successives par conversation_id, d'où un accumulateur persistant (F3).
- **D, Diagnostic** : l'unité est l'acte terminal, le seul que l'infirmier valide ou écarte. C'est donc l'objet du garde-fou (F5).
- **X, Transversal** : ce n'est pas un niveau supplémentaire mais une dimension de lecture. L'étiquette configuration permet de comparer les autres niveaux entre les quatre déploiements.

Les flèches traduisent la causalité : les tours alimentent la conversation, qui se clôture par un diagnostic, lui-même comparé entre configurations.

Traçabilité : du besoin à la mesure

Chaque partie prenante identifiée au §1.2 se voit attribuer au moins une question opérationnelle, et chaque question une métrique précise. Un besoin sans mesure reste ainsi visible, au lieu d'être silencieusement oublié.

Partie prenante	Question posée	Mesure
Personnel infirmier	L'assistant répond-il assez vite ?	R1 / D2
Personnel infirmier	Les patients abandonnent-ils l'entretien ?	C5
Équipe IA / data	Le format du diagnostic dérive-t-il ?	D1
Équipe IA / data	Combien d'inférences échouent ?	R10 / C6
Responsable conformité	Un diagnostic contient-il un langage absolu ou une donnée personnelle ?	D8 / D9
Décideur	Quelle configuration offre le meilleur compromis coût / latence ?	X1 / X2
Jury	Chaque besoin est-il mesuré ou justifié comme reporté ?	Statut de la carte

2. Audit de la solution data existante

Le système audité, ci-après désigné « le système existant », est pleinement en place : il ne s'agit donc pas d'auditer une solution proposée sur le papier, mais un système réel, déjà validé, que l'on observe depuis l'extérieur sans jamais en modifier le code.

2.1 Solution actuelle et proposée

Le système existant suit une architecture hexagonale (ports et adaptateurs), qui isole la logique métier de ses dépendances techniques. Cette discipline a été reprise à l'identique côté audit, afin que les deux projets restent cohérents et que l'audit lui-même reste testable.

Architecture actuelle

Le flux applicatif part de l'interface infirmier, traverse une API qui orchestre les cas d'usage métier, puis atteint le moteur d'inférence choisi. Chaque échange et chaque diagnostic produit sont consignés dans un journal d'audit structuré, qui constitue la seule trace exploitable du comportement réel du système.

- **Interface** : application Streamlit utilisée par le personnel infirmier.
- **Orchestration** : API REST exposant la création de conversation, l'envoi de messages et la demande de diagnostic.
- **Inférence** : moteur interchangeable selon la configuration (llama.cpp quantifié ou Transformers en pleine précision).
- **Traçabilité** : un journal d'audit au format structuré, écrit à chaque tour d'entretien et à chaque diagnostic.

Outils/technos utilisés

La pile technique du système existant se résume comme suit :

Composant	Technologie retenue
Modèle	Qwen3-1.7B, affiné SFT puis DPO, avec adaptateur LoRA
Inférence quantifiée	llama.cpp, quantification Q4_K_M, exécution CPU
Inférence pleine précision	Transformers + PEFT, bf16, CPU (usage comparatif)
API applicative	FastAPI
Interface utilisateur	Streamlit
Gestion de paquets	uv
Qualité / tests	pytest, ruff
Architecture	Hexagonale (ports et adaptateurs)

Le moteur étant interchangeable, chaque configuration (CHSA_MOTEUR_INFERENCE) écrit son propre journal, audité séparément.

Configuration	Moteur / matériel	Coût
local	llama.cpp Q4_K_M, CPU	0 €
comparaison_precision_cpu	Transformers bf16, CPU	0 €
distant (Space)	vLLM, GPU L4	~0,80 $/h
distant (HF Job)	vLLM, GPU L4, 30 min	Par job

Les jobs d'entraînement et d'évaluation, qui ne produisent aucun journal, sont hors périmètre.

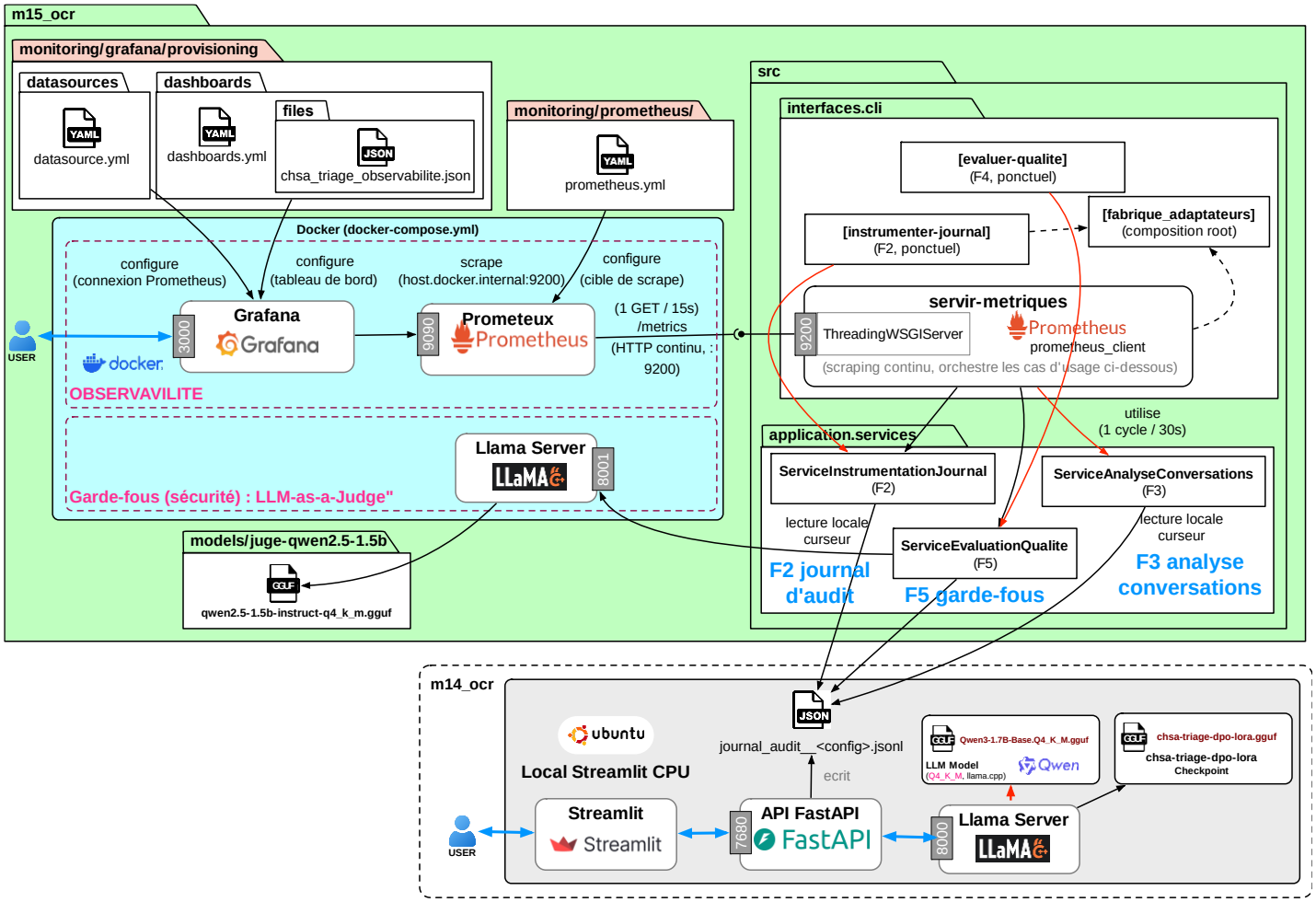


Figure 2. Composants du système existant et du dispositif d'audit, avec leurs ports (API, journal, interfaces HTTP), aucun composant d'audit n'est importé par le système existant.

Lecture de la Figure 2 : une dépendance à sens unique. Le cadre pointillé m14_ocr délimite le système audité, laissé intact. Toutes les flèches qui le relient au dispositif m15_ocr sont des lectures : le fichier journal est lu via un curseur, et les ports HTTP sont interrogés en local. Aucune flèche ne va dans l'autre sens. Le système audité ignore l'existence de l'audit. On pourrait retirer entièrement m15_ocr sans qu'une seule ligne de m14_ocr cesse de fonctionner : c'est la traduction architecturale de la contrainte « inchangé à l'octet près ».

2.2 Évaluation de l'adéquation aux besoins

Cette évaluation confronte le système existant aux besoins identifiés au chapitre précédent, selon six critères usuels d'audit d'une solution data/IA.

Critères d'analyse

Six critères usuels d'audit d'une solution data/IA ont servi de grille de lecture commune, pour comparer le système existant aux besoins identifiés sans biais de confirmation :
Le tableau suivant synthétise, critère par critère, le constat établi par l'audit :

Critère	Constat de l'audit
Performance	Latence mesurée précisément autour du seul appel d'inférence réel, ce qui constitue une bonne pratique déjà en place côté système existant.
Robustesse	Les échecs d'inférence sont déjà journalisés, mais jamais agrégés ni rendus visibles : un échec reste aujourd'hui invisible tant qu'on ne lit pas le fichier journal.
Sécurité	Accès protégé par clé d'API ; aucune donnée personnelle persistée dans le journal d'audit.
Coût	Quatre configurations aux profils de coût très différents (calcul local gratuit contre hébergement GPU payant), sans aucune donnée consolidée permettant de les comparer objectivement.
Maintenance	Architecture hexagonale testée, qui offre une base saine pour faire évoluer le système sans dette technique excessive.
Monitoring	Absent. Il s'agit d'une décision de périmètre assumée et documentée dans le code même du système existant : « jamais une stack d'observabilité lourde ».

Identification des écarts et limites.

Quatre écarts structurants ressortent de cette évaluation :

- Aucune vision agrégée, ni dans le temps ni entre les quatre configurations de déploiement.
- Le niveau d'urgence clinique (ESI) n'est jamais persisté dans le journal : il n'existe que de façon transitoire, le temps d'une réponse HTTP.
- Aucun seuil ni alerte ne permet de détecter une dérive de performance ou de fiabilité avant qu'un utilisateur ne la constate directement.
- Aucune donnée ne permet de comparer objectivement le coût réel des quatre configurations entre elles.

Visualisation des flux / processus existants.

Le schéma suivant détaille, pour un seul tour d'entretien, où et comment chaque donnée exploitée par l'audit est produite par le système existant : la mesure de latence n'entoure strictement que l'appel réseau réel vers le moteur d'inférence, jamais la construction de la requête ni l'enregistrement dans le journal.

Ce que mesure exactement latence_ms

Le chronomètre encadre le seul appel HTTP au moteur d'inférence. Il exclut la construction du prompt, l'écriture dans le journal et le trajet Streamlit → API. La latence auditée est donc celle du modèle, jamais celle perçue par l'infirmier. Mesurer l'écart entre les deux (X4) supposerait d'instrumenter le client HTTP lui-même.

Du journal aux métriques

Champ de l'entrée	Métrique dérivée
chaque entrée	chsa_journal_entrees_total
latence_ms	chsa_journal_latence_secondes
nombre_tokens_sortie	chsa_journal_tokens_sortie
format_respecte (diagnostic)	chsa_diagnostic_format_respecte_total
entrée d'échec	chsa_journal_erreurs_total
conversation_id	agrégats F3 (tours, temps, échecs)

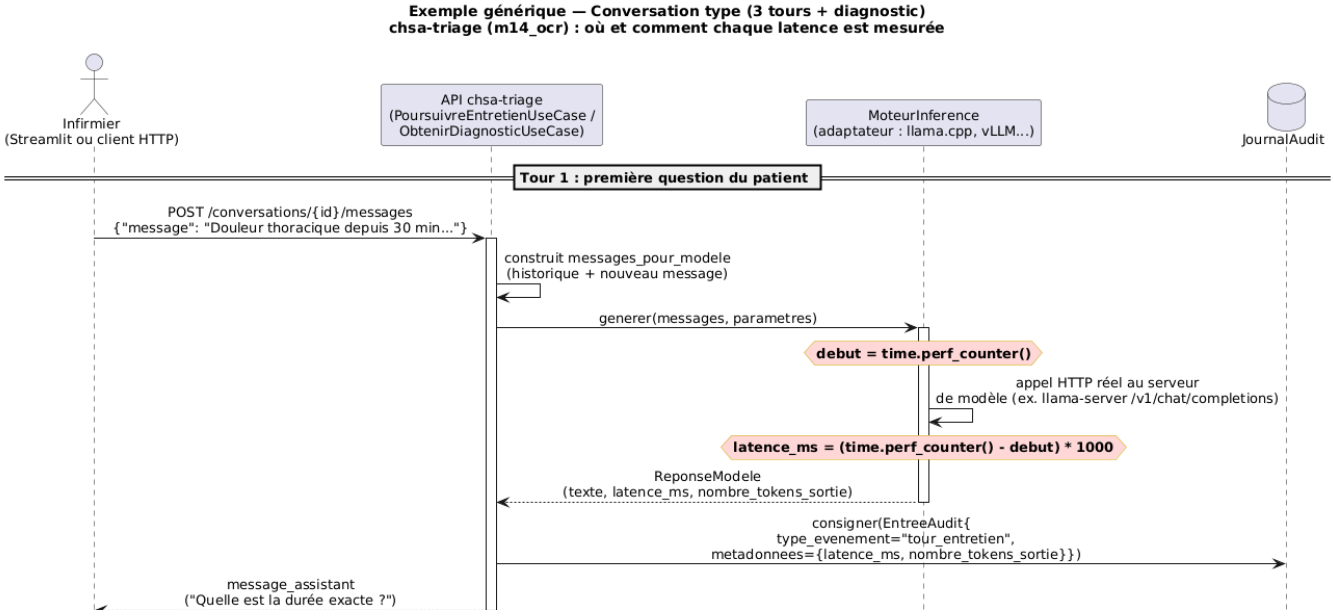


Figure 3. Déroulé d'un tour d'entretien dans le système existant, et point exact de mesure de la latence réelle d'inférence.

Le journal, source de vérité : garanties et limites

Le journal offre trois garanties utiles à l'audit : une écriture en ajout seul (*append-only*), jamais réécrite ; un horodatage UTC par entrée ; la version du modèle consignée à chaque entrée (version_modele).
Il présente aussi deux limites :

- sur le Space, il est **perdu sans volume monté**, sauf publication périodique vers un dataset Hugging Face, qui introduit un délai ;
- le texte saisi par l'infirmier y figure **tel quel, sans anonymisation** (M14, §5).

Conséquences pour la conception de l'audit

Ces propriétés ont imposé trois règles :

- **Lecture incrémentale par curseur** : chaque entrée n'est comptée qu'une fois, même après un redémarrage de l'audit.
- **Aucun texte dans les métriques** : seuls des comptages et des durées sont exportés vers Prometheus, jamais entree ni sortie.
- **Seule exception, le juge-LLM (F5)** : il lit le texte du diagnostic, mais localement. Ce texte ne quitte jamais la machine.

3. Identification d'une solution technique cible

Ce chapitre présente la solution retenue pour combler les écarts identifiés au chapitre précédent : comparatif des approches envisagées, schéma d'architecture, justification technologique, puis méthodologie de priorisation des cas d'usage restants.

Comparatif d'approches techniques

La solution cible repose sur trois décisions structurantes. Chacune a été tranchée par comparaison directe avec au moins deux alternatives concrètes, jamais par défaut. Le tableau suivant synthétise les options envisagées et le choix retenu ; leurs avantages et inconvénients sont détaillés juste après.

Décision	Approches comparées	Choix retenu
Pile d'observabilité	SaaS managé (Datadog, Grafana Cloud) · tableau de bord applicatif ad hoc (Streamlit) · Prometheus + Grafana auto-hébergés	Prometheus + Grafana, conteneurisés localement
Garde-fou qualité/sécurité	Règles/regex sur le texte · API LLM distante (OpenAI, Anthropic) comme juge · LLM local dédié comme juge	LLM local dédié (juge-LLM), jamais d'API distante
Modèle du juge-LLM	Phi-3-mini-4k-instruct (3.8B, Q4, 2.4 Go) · Qwen2.5-1.5B-Instruct (1.5B, Q4, 1.1 Go)	Qwen2.5-1.5B-Instruct

Pourquoi pas Phi-3-mini essayé en premier (meilleure réputation de raisonnement à cette taille), mais sa consommation réelle a dépassé 4 Go de RAM lors du chargement sur l'environnement cible (plafond de 6 Go), au point de devoir être interrompu avant saturation mémoire. Qwen2.5-1.5B se stabilise à 1.8 Go, seul le second a été retenu, sur preuve empirique et non sur réputation.

Contrat du juge-LLM

Le juge reçoit **le seul texte du diagnostic**, jamais la conversation complète : il voit moins de données et ne peut pas être influencé par l'historique. Il renvoie un **score sur 10 et une justification** (ResultatEvaluationQualite). Un diagnostic dont le score passe sous le seuil configuré déclenche une alerte comptée (chsa_garde_fou_alerte_total), jamais un blocage.

Avantages/inconvénients.

Prometheus + Grafana auto-hébergés :

- ✅ Gratuit, aucune donnée de santé transmise à un tiers (contrainte RGPD du secteur).
- ✅ Format d'exposition standard, réutilisable par n'importe quel outil compatible (Alertmanager, Thanos…).
- ✅ Cohérent avec le reste du projet : aucune dépendance cloud nouvelle.
- ❌ Pas de haute disponibilité ni de rétention longue durée par défaut (acceptable pour un POC, à revoir en production).
- ❌ Alerting (Alertmanager) non configuré à ce stade.

Juge-LLM local dédié :

- ✅ Aucune donnée de diagnostic ne quitte la machine, exigence non négociable en contexte hospitalier.
- ✅ Gratuit, latence prévisible, pas de dépendance réseau externe.
- ✅ Modèle distinct de celui audité : élimine le biais d'auto-évaluation.
- ❌ Capacité de raisonnement limitée (1.5B paramètres) : justifications parfois incohérentes même quand le score est correct.
- ❌ Garde-fou rétrospectif, jamais préventif (voir §4.2, impacts et limites).

Schéma d'architecture cible clair

L'architecture cible reconduit la discipline hexagonale (ports et adaptateurs) déjà en place dans le système audité, appliquée cette fois au dispositif d'audit lui-même. Un domaine métier sans dépendance externe, des cas d'usage qui orchestrent des ports, et des adaptateurs concrets interchangeables, Prometheus pour l'export de métriques, un lecteur de journal JSONL pour la source de vérité, un llama-server local pour le juge-LLM.

Schéma complet (composants, flux, étiquetage de sécurité) : docs/diagrams/architecture_hexagonale.svg. Trois frontières de communication, toutes HTTP locales ou lecture de fichier local, aucune jamais distante :

- Lecture du journal d'audit : fichier JSONL local, jamais un appel réseau.
- Vérification de disponibilité et export de métriques : HTTP vers localhost uniquement.
- Appel au juge-LLM : HTTP vers un second llama-server local et dédié, port distinct de celui du système audité.

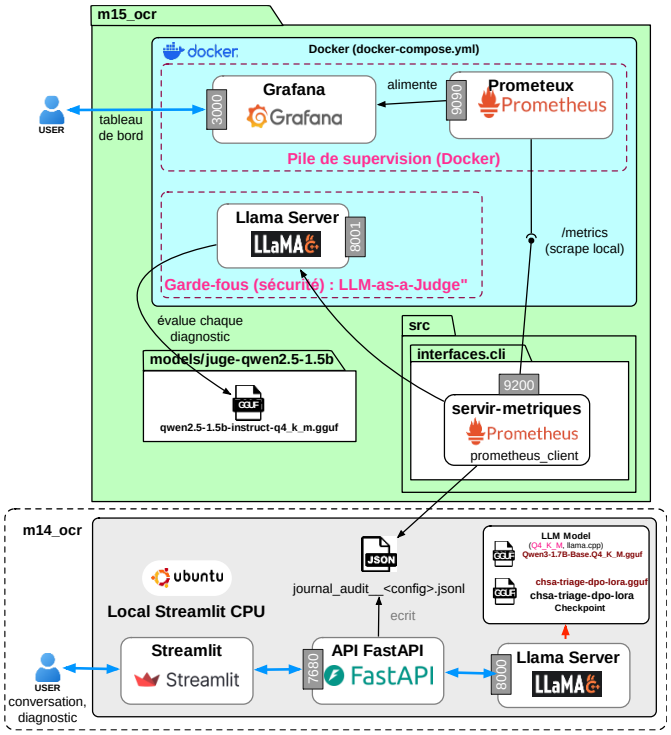


Figure. Architecture cible résumée : trois systèmes (audité, observabilité, supervision), zéro frontière distante (docs/diagrams/architecture_cible.svg).

Du garde-fou prévu (M14) au garde-fou livré (F5)

Le cahier des charges du système audité prévoyait un garde-fou NF4, resté non implémenté (M14, §6). Le garde-fou F5 n'en est pas l'implémentation : il en est une version volontairement plus restreinte.

Aspect	NF4 prévu (M14)	F5 livré (M15)
Moment	Avant l'infirmier	Après coup
Effet	Rejet (score forcé à 0)	Alerte, sans blocage
Échelle	1 à 7, seuil 4	0 à 10, seuil configurable
Juge	Non choisi	Qwen2.5-1.5B local dédié
Périmètre	Sécurité clinique	Forme et sécurité

Le critère d'acceptation « aucune réponse < 4/7 » du M14 reste donc non atteint. F5 fournit en revanche la mesure qui manquait pour le calibrer : la part réelle de diagnostics sous le seuil.

Justification des choix technologiques

Chaque choix technologique répond à une contrainte concrète du contexte (hospitalier, local, sans budget d'infrastructure), jamais à une préférence par défaut.

Choix	Justification
API	**Format compatible OpenAI** (POST /v1/chat/completions) exposé nativement par llama-server, aucun client HTTP sur mesure à écrire ni maintenir.
Conteneurisation	**Limitée à Prometheus et Grafana** (stateless côté logique métier, bénéficient réellement d'une isolation et d'un redémarrage propre). Les services Python restent des processus hôte : ils ont besoin d'un accès direct au système de fichiers local pour lire le journal d'audit, ce qu'un conteneur rendrait seulement plus complexe sans bénéfice.
Base de données	**Aucune ajoutée** : la base de séries temporelles native de Prometheus suffit au périmètre (compteurs et histogrammes), évite une dépendance opérationnelle supplémentaire.
Orchestrateur	**Aucun (pas de Kubernetes/Nomad)** : deux conteneurs et trois processus Python ne justifient pas la charge opérationnelle d'un orchestrateur à l'échelle d'un POC.
LLM (juge)	**Qwen2.5-1.5B-Instruct**, quantifié Q4_K_M, servi par llama.cpp, même moteur d'inférence que la configuration CPU gratuite déjà en production côté système audité, donc aucune nouvelle compétence d'exploitation à acquérir.

Mode d'exposition des métriques

Prometheus fonctionne en mode *pull* : c'est lui qui vient lire les métriques, toutes les 15 secondes, sur le port 9200 du processus servir-metriques. Celui-ci relit le journal toutes les 30 secondes. L'audit ne dépend donc jamais de la disponibilité de Prometheus. Comme les métriques sont des compteurs cumulatifs, un relevé manqué fait perdre de la résolution temporelle, mais jamais de comptage.

Ce double intervalle a une conséquence : une métrique peut avoir jusqu'à 45 secondes de retard sur l'événement. C'est acceptable pour une supervision rétrospective, mais pas pour une alerte temps réel. Les commandes ponctuelles (instrumenter-journal, analyser-conversations) offrent un second mode d'exposition : elles écrivent des fichiers .prom horodatés, utiles pour une analyse reproductible hors ligne.

Une limite découle de ce choix. Les compteurs vivent en mémoire, alors que le curseur de lecture est persisté sur disque. Un redémarrage de servir-metriques remet donc les totaux bruts à zéro sans recompter les entrées déjà lues. Les panneaux fondés sur increase() ou rate() absorbent cette remise à zéro ; un panneau de total brut, en revanche, la montre.

Méthodologie d'identification, d'évaluation et de priorisation des cas d'usage

L'audit du système existant (chapitre 2) a fait ressortir quatre écarts structurants. Chaque cas d'usage de métrique a été classé selon trois états de maturité, avant d'être priorisé :

État	Définition	Critère de passage à l'état suivant
Construit	Métrique Python réelle, testée, exposée en production sur ce dépôt.	-
Dérivable	Donnée déjà disponible dans les métriques existantes ; seule une requête PromQL ou un panneau Grafana manquent, aucun code Python nouveau.	Ajout d'un panneau Grafana seul.
Indisponible	Nécessite une donnée jamais consignée par le système audité (ex. niveau ESI), ou un composant jamais construit (évaluation RAGAS par lots).	Nouveau développement applicatif.

Cette classification, détaillée métrique par métrique en annexe, a directement guidé l'ordre de construction : F0 (connectivité) → F1 (métriques) → F2 (journal) → F3 (conversations) → F5 (garde-fous), toujours le prochain cas d'usage dérivable à moindre effort et à plus forte valeur de pilotage, jamais le plus simple pris isolément.

Procédure de classement

Chaque métrique de la carte a été soumise à la même suite de questions, dans cet ordre :

- La donnée figure-t-elle dans une entrée du journal ? Si non, la métrique est **indisponible**.
- Si oui, une métrique Prometheus existante la porte-t-elle déjà ? Si oui, elle est **dérivable** par une simple requête PromQL.
- Sinon, elle est **à construire** en Python.

Un quatrième état, absent de la classification initiale, est apparu en cours de route : **écarté**, pour une métrique redondante avec une autre déjà construite. C'est le cas de la durée totale de conversation (C4), qui double le temps jusqu'au diagnostic (C2). La valeur de pilotage, quant à elle, a été jugée à un critère simple : la métrique répond-elle à une question posée par une partie prenante identifiée au §1.2 ?

Hiérarchisation documentée des cas d'usage

Matrice valeur / effort appliquée aux quatre écarts identifiés au chapitre 2 et aux groupes de métriques associés :

	Effort faible	Effort élevé
Valeur élevée	Fait, Vision agrégée entre configurations (X1) ; Fait, Garde-fou sécurité/forme (D8/D9, F5)	Prochain, Niveau ESI persisté (F4) ; Différé, Pertinence clinique réelle (C7)
Valeur faible	Dérivable, Tokens/seconde, position du tour (R3, R7) ; Dérivable, Carte de chaleur d'activité (X3)	Écarté, Pertinence RAGAS par lots (R6) ; Écarté, Dégénérescence/boucle (R8)

Cette matrice traduit visuellement l'ordre de construction effectivement suivi : les cas d'usage à forte valeur et faible effort ont systématiquement été traités en premier, qu'ils soient déjà livrés (garde-fous) ou encore à engager (niveau ESI).

Le cas du niveau ESI

Le niveau ESI est classé « prochain » malgré un effort élevé, et c'est le seul cas d'usage qui entre en conflit direct avec la contrainte de non-modification du système audité. Deux voies existent pour le lever.
La première consiste à instrumenter l'API du système audité pour qu'elle persiste le niveau. C'est la voie propre, mais elle suppose de rouvrir un projet déjà validé.

La seconde consiste à extraire le niveau du texte brut des diagnostics déjà consignés, lorsque leur format est respecté. Cette voie est réalisable sans toucher au système, mais biaisée : elle ne mesurerait que les diagnostics bien formés, précisément ceux qui posent le moins de problèmes.
Enfin, une limite de méthode : cette matrice a été cotée par un développeur unique, sans validation croisée des parties prenantes. Elle traduit donc un ordre de travail argumenté, pas une priorisation clinique.

4. Stratégie de mise en œuvre et d'industrialisation

Ce chapitre détaille comment la solution cible a été, et continuera d'être, mise en œuvre : séquencement des phases, outillage retenu à chaque étape, puis les éléments d'aide à la décision (risques, budget, indicateurs de succès, impacts) nécessaires au pilotage du projet.

4.1 Proposition de démarche projet

Roadmap de mise en œuvre

Le tableau suivant retrace les phases réellement livrées à ce jour, avec leur jalon, leur responsable et leur état d'avancement ; la représentation visuelle qui suit en donne une lecture chronologique directe.

Phase	Jalon	Responsable	État
F0, Connectivité	Vérification des dépendances externes (HF Hub, API auditée, MLflow)	Solo (dév. unique)	Livré
F1, Métriques	Export Prometheus générique (compteurs, histogrammes)	Solo	Livré
F2, Journal d'audit	Instrumentation du journal JSONL, scraping continu	Solo	Livré
F3, Conversations	Agrégats niveau conversation (tours, temps, échecs)	Solo	Livré
F5, Garde-fous	Juge-LLM dédié, score et alerte de sécurité/forme	Solo	Livré
F4, Niveau ESI	Instrumentation directe de l'API pour persister l'ESI	À planifier	Non démarré

F4 porte ce numéro depuis la conception initiale du projet (instrumentation ESI) ; le garde-fou a reçu le numéro F5 pour ne jamais réutiliser une étiquette déjà réservée.

Découpage en étapes

Chaque phase (F0 à F5) a suivi systématiquement la même séquence, illustrée ici par la phase F5, la plus récemment livrée :

- **Développement**, nouveau port (EvaluateurQualite), nouvel adaptateur (LlamaServerEvaluateurQualite), nouveau cas d'usage (ServiceEvaluationQualite), sans jamais modifier le système audité.
- **Tests**, 7 tests unitaires sur le cas d'usage avec doublures en mémoire, exécutés avant toute intégration réelle.
- **Conteneurisation**, aucune nécessaire pour le code Python (processus hôte) ; seul le panneau Grafana a été ajouté au conteneur existant.
- **Intégration**, câblage dans servir-metriques, le point d'entrée qui orchestre déjà F0 à F3.
- **Déploiement, choix et déploiement du modèle juge**, avec un premier incident réel (dépassement mémoire de Phi-3-mini) et une correction mesurée avant nouvelle tentative.
- **Monitoring, vérification en conditions réelles** : deux diagnostics contrastés soumis au juge (score 8/10 contre 0/10), puis rejeu automatique de l'historique complet du journal (14 évaluations réelles, 5 alertes) dès la remise en service du juge.

Lecture de l'architecture détaillée

La figure ci-dessous rend visible ce découpage par tranches verticales : chaque étiquette de phase (F0 à F5) recouvre un cas d'usage, les ports qu'il consomme et les adaptateurs qui les implémentent. Elle montre surtout que le coût marginal de chaque phase a décru. F2, F3 et F5 réutilisent tous les deux mêmes ports, LecteurJournalAudit et ExportateurMetriques, et chaque nouvelle phase n'a introduit qu'un seul port supplémentaire, au plus : AccumulateurConversations pour F3, EvaluateurQualite pour F5.

Le garde-fou, pourtant la fonctionnalité la plus complexe, s'est ainsi réduit à un port, un adaptateur et un cas d'usage. La phase F4 apparaît, elle, comme un emplacement vide dans la couche application. Elle n'a ni classe ni port, ce qui confirme qu'elle exigerait une donnée que le journal ne fournit pas, et non un simple ajout de code.

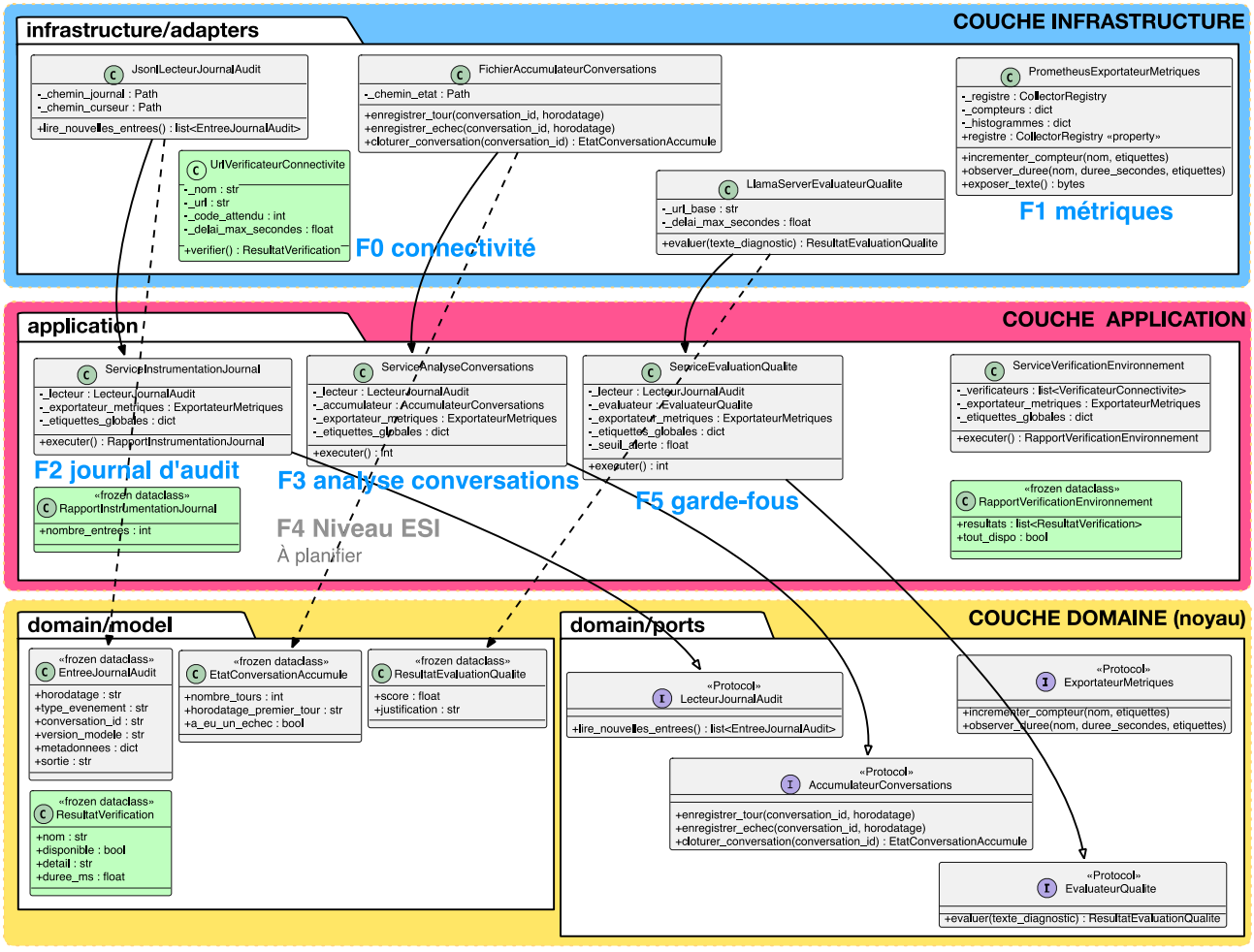


Figure : Architecture Hexagonal detaillée.

Outils envisagés pour chaque phase

Chaque outil a été retenu pour sa cohérence avec le reste de la pile existante, jamais introduit isolément : aucune nouvelle brique d'infrastructure n'a été ajoutée sans réutiliser un choix déjà en place côté système audité ou côté observabilité.

Phase	Outils
Développement	uv (gestion de dépendances), architecture hexagonale, httpx
Tests	pytest, doublures en mémoire, ruff (lint)
Conteneurisation	Docker Compose (Prometheus, Grafana uniquement)
Intégration	servir-metriques (composition root partagée, scraping continu)
Déploiement	llama.cpp / llama-server, modèles GGUF quantifiés (Hugging Face)
Monitoring	Prometheus (séries temporelles), Grafana (tableaux de bord provisionnés)

4.2 Aide à la prise de décision

Synthèse des risques et opportunités :

Risque	Probabilité observée	Levier d'atténuation appliqué
Saturation mémoire (OOM) en environnement contraint (6 Go)	Élevée, survenue trois fois en cours de projet	Surveillance active de la RAM à chaque lancement de modèle, seuil d'abandon automatique, choix d'un modèle plus petit sur preuve empirique
Latence d'inférence CPU incompatible avec un usage temps réel	Élevée, appels de diagnostic dépassant 120 s sur ce matériel	Documenté comme limite connue ; configurations GPU distantes restent l'option de production
Faible capacité de raisonnement du juge-LLM (1.5B paramètres)	Confirmée, justifications parfois incohérentes malgré un score correct	Périmètre du juge volontairement restreint à la forme/sécurité, jamais à l'exactitude clinique
Garde-fou rétrospectif seulement (pas de blocage temps réel)	Structurelle, assumée	Documenté explicitement partout (code, diagrammes, rapport) pour ne jamais être présenté comme préventif

Opportunité symétrique : chaque incident mémoire a produit une preuve réutilisable (seuils de sécurité, choix de modèle) directement intégrée au code et à la documentation.

Scénarios budgétaires :

Scénario	Coût d'inférence	Coût DevOps/infra
Retenu, llama.cpp local (triage + juge)	0 € (CPU local)	0 € (aucun service managé)
Alternatif, vLLM sur GPU distant (Space HF)	Facturation horaire GPU tant que le service tourne (panneau Grafana « Coût estimé », X2, déjà construit)	0 € (Space HF géré)
Alternatif, juge via API LLM distante	Facturation par appel/token	Exclu : transmission de texte de diagnostic à un tiers, incompatible avec le contexte hospitalier

Indicateurs de succès (KPI) :

KPI techniques :

- **Taux d'erreur d'inférence** (chsa_journal_erreurs_total / chsa_journal_entrees_total), cible < 5 %
- **Conformité du format de diagnostic** (chsa_diagnostic_format_respecte_total), cible > 98 %
- **Latence p95 de réponse** (chsa_journal_latence_secondes)

KPI business :

- **Taux de diagnostics sous le seuil de garde**-fous (chsa_garde_fou_alerte_total), signal de supervision, pas un blocage
- **Coût cumulé** comparé entre configurations (panneau X2)
- Taux **d'abandon de conversation** (C5, aujourd'hui non disponible, premier chantier F4 complémentaire)

Impacts potentiels et leviers d'atténuation :

Impact	Nature	Levier appliqué ou recommandé
Transmission de données de santé à un tiers	Légal/réglementaire (RGPD, secret médical)	Aucune API distante : audit, métriques et juge-LLM restent 100 % locaux
Biais d'un petit modèle juge (1.5B)	Organisationnel/qualité	Périmètre volontairement restreint à la forme, jamais à l'exactitude clinique ; documenté explicitement
Absence de clé API sur les endpoints locaux	Sécurité	Acceptable pour un POC sur hôte local ; recommandé avant tout accès réseau élargi
Latence CPU incompatible avec l'urgence clinique	Business/organisationnel	Configuration CPU réservée aux environnements de test ; configuration GPU recommandée en production
Garde-fou perçu à tort comme préventif	Organisationnel (confiance excessive)	Mention explicite « **rétrospectif, pas préventif** » dans le code, les diagrammes et ce rapport

5. Contrôle et suivi du projet

Ce chapitre décrit d'abord comment le projet lui-même a été piloté, puis comment la solution livrée est surveillée une fois en production, à travers les mêmes outils qu'elle expose.

5.1 Tableau de bord de pilotage

Indicateurs de suivi

Cinq dimensions usuelles de suivi de projet, confrontées aux faits réels plutôt qu'à une estimation :

Dimension	Constat réel
Délais	Première livraison (F0-F3 + pile Prometheus/Grafana) le 22/10/2026 ; garde-fous (F5) et diagrammes à jour le 03/11/2026, deux jalons livrés, aucun retard déclaré faute de délai contractuel fixé (projet personnel).
Coûts	0 € : inférence 100 % locale (llama.cpp CPU), aucun service managé payant, aucune API tierce facturée.
Livrables	2 932 lignes de code source, 46 tests automatisés, 9 diagrammes UML (svg/png/pdf), 12 panneaux Grafana en production, chapitres 1 à 4 du présent rapport.
Qualité des données	Journal d'audit réel exploité : 73 tours d'entretien, 15 diagnostics, 6 échecs d'inférence, aucune donnée simulée ou fabriquée pour les métriques de production.
Performances	46/46 tests passent, lint sans erreur ; vérification de bout en bout en conditions réelles (score du juge-LLM 8/10 vs 0/10 sur deux cas contrastés, 14 évaluations historiques rejouées automatiquement).

Méthodologie de gestion

Projet personnel à développeur unique : pas de cérémonie Scrum/Kanban formelle. La gestion suit une livraison itérative par tranche verticale (F0 → F1 → F2 → F3 → F5), chaque tranche traversant systématiquement développement → tests → intégration → déploiement → monitoring avant de passer à la suivante (détail §4.1). L'historique Git tient lieu de journal de bord ; chaque commit correspond à une tranche livrée et vérifiée, jamais à un état intermédiaire non testé.

5.2 Outils et process de suivi

Outils de suivi en production

Prometheus (collecte, base de séries temporelles) et Grafana (visualisation, tableaux de bord provisionnés automatiquement), RAGAS envisagé mais non implémenté (voir indicateur R6 ci-dessous). Inventaire complet des indicateurs du domaine, construits, dérivables ou indisponibles :

État Construit

#	Indicateur	Source réelle (nom exact)	Qui la génère
R9	Volume de requêtes	chsa_journal_entrees_total	ServiceInstrumentationJournal (F2), panneau n°1
R1/D2	Latence réponse / diagnostic (p50/p95)	chsa_journal_latence_secondes	ServiceInstrumentationJournal (F2), panneau n°2
R10	Taux d'échec d'inférence	chsa_journal_erreurs_total	ServiceInstrumentationJournal (F2), panneau n°3
D1	Respect du format de diagnostic	chsa_diagnostic_format_respecte_total	ServiceInstrumentationJournal (F2), panneau n°4
C6	Taux d'échec par conversation	chsa_conversation_avec_echec_total	ServiceAnalyseConversations (F3), panneau n°5
C1	Nombre de tours avant diagnostic	chsa_conversation_nombre_tours	ServiceAnalyseConversations (F3), panneau n°6
R2/D3	Tokens de sortie (p50/p95)	chsa_journal_tokens_sortie	ServiceInstrumentationJournal (F2), panneau n°7
X2	Coût estimé cumulé	Formule Grafana pure sur chsa_journal_tokens_sortie_sum	Panneau n°8, variable precio_1k_tokens
C2	Temps jusqu'au diagnostic	chsa_conversation_temps_diagnostic_secondes	ServiceAnalyseConversations (F3), panneau n°9
C3	Taux de reformulation	chsa_conversation_reformulation_total	ServiceAnalyseConversations (F3), panneau n°10
D8	Score garde-fous (juge-LLM)	chsa_garde_fou_score	ServiceEvaluationQualite (F5), panneau n°11
D9	Alertes garde-fous	chsa_garde_fou_alerte_total	ServiceEvaluationQualite (F5), panneau n°12
X1	Comparaison entre configurations	Étiquette configuration déjà posée sur chaque métrique	N'importe quel service + by (configuration) en PromQL

État Dérivable :

#	Indicateur	Source réelle (nom exact)	Qui la génère
-	Longueur de conversation (nb d'entrées)	Regrouper chsa_journal_entrees_total par conversation_id et compter	Travail nouveau dans ServiceInstrumentationJournal
R3	Tokens par seconde	PromQL pur (tokens/latence)	Panneau Grafana seul
R7	Position du tour dans la conversation	PromQL pur : count() par conversation_id	Panneau Grafana seul
X3	Carte de chaleur de l'activité	chsa_journal_entrees_total dans le temps (série continue depuis servir-metriques)	Panneau Grafana heatmap seul

État Écartée :

#	Indicateur	État	Source réelle (nom exact)	Qui la génère
C4	Durée totale de conversation	Écartée	Redondante avec C2	Non retenue délibérément

État Indisponible :

R6	Pertinence (RAGAS / juge)	Nécessiterait un job d'évaluation par lots séparé (RAGAS)	Hors périmètre, ne pas confondre avec D8/D9
C5	Taux d'abandon	Nécessite de détecter l'absence d'un événement futur	Fenêtre glissante à construire
C7	Efficacité clinique du diagnostic	Nécessiterait un référentiel médical de vérité terrain	Hors périmètre d'un POC
R4/R5	Longueur et pertinence heuristique de la question	Jamais consigné par le système existant	-
R8	Dégénérescence / boucle	Aucune détection construite	-
D4	Niveau ESI	Jamais persisté dans le journal (réservé F4)	Instrumentation directe de l'API, jamais démarrée
D5/D6	Longueur du raisonnement / distribution ESI	Dépend de D4	-
D7	Cas ESI urgents et leur latence	Dépend de D4	Nécessite F4
X4	Latence inférence pure vs HTTP totale	Nécessiterait d'instrumenter le client HTTP lui-même	-

12 indicateurs construits et exposés en production (panneaux Grafana 1-12) · 5 dérivables sans code Python nouveau · 8 indisponibles, chacun avec sa cause exacte.

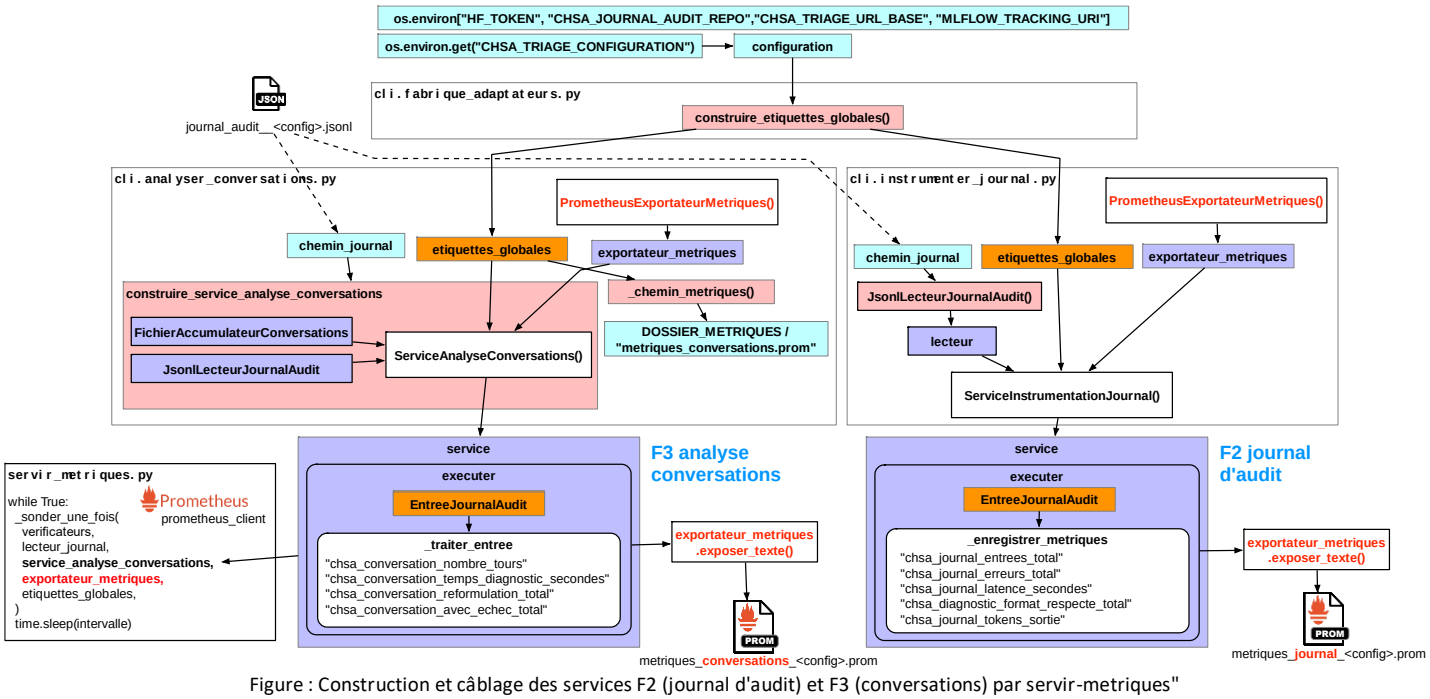
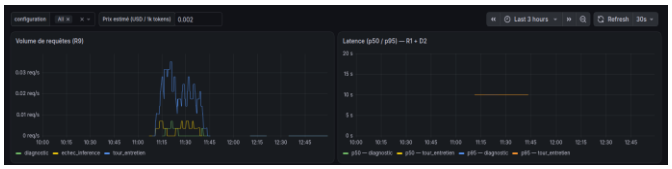


Figure : Construction et câblage des services F2 (journal d'audit) et F3 (conversations) par servir-metriques"



Figure, Tableau de bord, rangée 1 : volume de requêtes (R9) et latence p50/p95 (R1+D2), données réelles sur 7 jours.

- **Volume de requêtes (R9) :** 122 tours d'entretien et 20 diagnostics traités sur la fenêtre de 3 heures ; le pic correspond exactement à la génération de trafic.
- **Latence (p50/p95), R1+D2 :** latence moyenne réelle de 20,0 s par tour et 33,1 s par diagnostic, cohérent avec une inférence CPU sans accélération GPU ; le plateau à droite montre le juge-LLM traitant l'arriéré juste après.



Figure, Tableau de bord, rangée 2 : taux d'échec, conformité du format, échec par conversation, tours, temps, reformulation.

- **Taux d'échec d'inférence (R10)** : 0 % instantané (aucun échec dans les 15 dernières minutes), mais 24 échecs réels au total sur la fenêtre (timeout à 120 s côté système audité, limite connue, non corrigée volontairement).
- **Respect du format de diagnostic (D1)** : 0 %, constat le plus notable : les 20 diagnostics produits n'ont jamais respecté le format structuré attendu, un signal réel côté système audité, pas un artefact de m15_ocr.
- **Taux d'échec par conversation (C6)** : 5 % (1 conversation sur 20).
- **Nombre de tours avant diagnostic (C1)** : médiane de 4,3 tours.
- **Temps jusqu'au diagnostic (C2)** : médiane de 10,0 s, très inférieure à la moyenne réelle (89 s), quelques conversations ralenties par la contention CPU tirent la moyenne sans affecter la médiane.
- **Taux de reformulation (C3)** : aucune reformulation sur cette fenêtre.



Figure, Tableau de bord, rangée 3 : tokens de sortie (R2+D3) et coût estimé indicatif (X2).

- **Tokens de sortie (R2+D3)** : 86 tokens en moyenne pour un diagnostic contre 46 pour un tour d'entretien simple, cohérent avec un texte plus structuré.
- **Coût estimé (X2)** : environ 0,003 $ sur la fenêtre affichée (indicatif, 0,002 $/1000 tokens), négligeable, confirme l'intérêt d'une configuration CPU locale gratuite face à une configuration GPU facturée à l'heure.



Figure, Tableau de bord, rangée 4 : score garde-fous (D8) et taux d'alerte (D9, F5), 35,7 % des diagnostics évalués sous le seuil.

- **Score garde-fous, juge-LLM (D8)** : score moyen de sécurité/forme de 5,3/10 sur les 18 diagnostics effectivement évalués (2 ont échoué par timeout).
- **Alertes garde-fous (D9)** : 7 diagnostics (38,9 %) sous le seuil d'alerte (5/10), principalement pour absence de recommandation de consulter un professionnel ou langage diagnostique trop absolu. *Rappel : ce score ne mesure jamais l'exactitude clinique, uniquement la forme et la sécurité du texte.*

Méthodologie de test et évaluation

- **Tests unitaires** : 46 tests pytest sur les cas d'usage, avec doublures en mémoire, exécutés avant toute intégration réelle.
- **Test API en conditions réelles (sandbox)** : conversations réelles envoyées à l'API du système audité (création de conversation, tours d'entretien, demande de diagnostic), sans aucune donnée simulée.
- **Vérification du juge-LLM** : deux diagnostics délibérément contrastés (un conforme, un violant les garde-fous) soumis en direct, scores 8/10 et 0/10 obtenus, alerte correctement déclenchée sur le second.
- **Tests de charge :** non réalisés à ce stade, limite connue, le matériel CPU utilisé en développement (2 threads) ne permet pas une mesure représentative d'un volume de production.

6. Conclusion & recommandations

Ce chapitre clôt le rapport : synthèse des décisions structurantes, limites assumées, et trajectoire recommandée pour la suite du projet.

Résumé des choix clés

- **Architecture hexagonale** reconduite côté audit, à l'identique du système audité, jamais une seule ligne de m14_ocr modifiée.
- **Chaîne 100 % locale**, y compris le garde-fou juge-LLM : aucune donnée de diagnostic ne quitte jamais la machine (contrainte non négociable en contexte hospitalier).
- **Garde-fou explicitement rétrospectif**, jamais présenté comme préventif, choix assumé plutôt que sur-promis.
- **Choix du modèle juge tranché sur preuve empirique** (incident mémoire réel), pas sur réputation.

Perspectives d'évolution.

- F4, **instrumentation directe de l'API** pour persister le niveau ESI (seul écart du chapitre 2 non encore traité).
- **Évaluation de pertinence par lots** (RAGAS, R6), distincte du garde-fou de sécurité/forme déjà livré.
- **Règles d'alerte Grafana/Alertmanager** sur les seuils déjà exposés (taux d'erreur, score garde-fous).
- **Panneau de carte de chaleur d'activité** (X3), donnée déjà disponible, ajout Grafana seul.

Prochaines étapes recommandées.

- Démarrer F4 (niveau ESI), seul écart du chapitre 2 non encore adressé.
- Ajouter des règles d'alerte sur les deux panneaux garde-fous déjà en production.
- Évaluer une configuration GPU pour la latence avant tout usage proche du temps réel.

7. Annexes (Références)

- **README.md** du dépôt, incluant la table complète des indicateurs ci-dessus et un résumé des chapitres 3-4.

- uv run verifier-environnement / servir-metriques / evaluer-qualite, points d'entrée CLI.
- docker compose up -d, pile Prometheus + Grafana.
- llama-server (modèle audité, port 8080) et llama-server (juge dédié, port 8001), lancement détaillé dans le README.

- **docs/diagrams/,** 9 diagrammes UML (architecture hexagonale, classes, composants, paquets, séquences, carte des métriques), chacun en .puml/.svg/.png/.pdf.
- **Tableau de bord Grafana en production**, (12 panneaux, données réelles).
- **Fichiers de configuration de la pile** : monitoring/prometheus/prometheus.yml (cible et intervalle de collecte) et monitoring/grafana/provisioning/ (source de données et tableau de bord chsa_triage_observabilite.json, provisionnés automatiquement).
- **Rapport technique M14** : rapport du système audité, référence pour toutes les mentions « (M14, §x) » du présent rapport.